



EBook Gratis

APRENDIZAJE Win32 API

Free unaffiliated eBook created from
Stack Overflow contributors.

#winapi

Tabla de contenido

Acerca de	1
Capítulo 1: Comenzando con la API de Win32	2
Observaciones	2
Versiones	2
Examples	2
Hola Mundo	2
Capítulo 2: Funciones de caracteres ansi- y anchos	5
Examples	5
Introducción	5
Capítulo 3: Gestión de archivos	7
Examples	7
Crea un archivo y escríbelo	7
Referencia API	7
Capítulo 4: Gestión de procesos y subprocessos	8
Examples	8
Crea un proceso y comprueba su código de salida	8
Crear un nuevo hilo	9
Capítulo 5: Informe de errores y manejo	10
Observaciones	10
Examples	10
Introducción	10
Error reportado solo por el valor de retorno	10
Error reportado con información adicional en caso de falla	11
Notas sobre la llamada a GetLastError() en otros lenguajes de programación	11
Idiomas .net (C #, VB, etc.)	11
lr	11
Error reportado con información adicional sobre el fracaso y el éxito	12
Error reportado como valor HRESULT	12
Convertir un código de error en una cadena de mensaje	13
Capítulo 6: Mensajes de ventana	15

Sintaxis.....	15
Examples.....	15
WM_CREATE.....	15
WM_DESTROY.....	15
WM_CLOSE.....	16
WM_SIZE.....	16
WM_COMMAND.....	17
Capítulo 7: Servicios de Windows.....	19
Examples.....	19
Compruebe si un servicio está instalado.....	19
Referencia API:.....	20
Capítulo 8: Subclase de Windows.....	21
Introducción.....	21
Sintaxis.....	21
Parámetros.....	21
Observaciones.....	21
Examples.....	22
Subclasificar el control de botones de ventanas dentro de la clase C ++.....	22
Manejo de mensajes de notificación de controles comunes dentro de la clase C ++.....	23
Capítulo 9: Tratar con ventanas.....	25
Examples.....	25
Creando una ventana.....	25
¿Qué es un mango?.....	28
Constantes.....	28
Tipos de Windows.....	29
Capítulo 10: Utilizando la documentación de MSDN.....	30
Introducción.....	30
Observaciones.....	30
Examples.....	30
Tipos de documentación disponibles.....	30
Encontrar documentación para una característica.....	30
Uso de la documentación de la función.....	31

Visión general	31
Sintaxis	31
Parámetros	31
Valor de retorno	31
Observaciones	31
Ejemplos	31
Requerimientos	31
Creditos	33

Acerca de

You can share this PDF with anyone you feel could benefit from it, downloaded the latest version from: [win32-api](#)

It is an unofficial and free Win32 API ebook created for educational purposes. All the content is extracted from [Stack Overflow Documentation](#), which is written by many hardworking individuals at Stack Overflow. It is neither affiliated with Stack Overflow nor official Win32 API.

The content is released under Creative Commons BY-SA, and the list of contributors to each chapter are provided in the credits section at the end of this book. Images may be copyright of their respective owners unless otherwise specified. All trademarks and registered trademarks are the property of their respective company owners.

Use the content presented in this book at your own risk; it is not guaranteed to be correct nor accurate, please send your feedback and corrections to info@zzzprojects.com

Capítulo 1: Comenzando con la API de Win32

Observaciones

WinAPI (también conocida como Win32; oficialmente llamada API de Microsoft Windows) es una interfaz de programación de aplicaciones escrita en C por Microsoft para permitir el acceso a las funciones de Windows. Los componentes principales de WinAPI son:

- WinBase: las funciones del kernel, CreateFile, CreateProcess, etc.
- WinUser: las funciones GUI, CreateWindow, RegisterClass, etc.
- WinGDI: las funciones gráficas, Ellipse, SelectObject, etc.
- Controles comunes: controles estándar, vistas de lista, controles deslizantes, etc.

Ver también:

- [Índice API de Windows](#) en MSDN.

Versiones

Las versiones de la API están vinculadas a la versión del sistema operativo. La documentación de MSDN especifica el sistema operativo mínimo soportado para cada función en la API.

Examples

Hola Mundo

Las aplicaciones de Microsoft Windows generalmente se escriben como una aplicación de consola o una aplicación de ventana (existen otros tipos, como servicios y complementos). La diferencia para el programador es la diferencia en la interfaz para el punto de entrada principal para la fuente de la aplicación proporcionada por el programador.

Cuando se inicia una aplicación C o C ++, el punto de entrada ejecutable utilizado por el [cargador ejecutable](#) es el Runtime que proporciona el compilador. El cargador ejecutable lee en el ejecutable, realiza cualquier corrección a la imagen necesaria y luego invoca el punto de entrada ejecutable que para un programa C o C ++ es el tiempo de ejecución proporcionado por el compilador.

El punto de entrada ejecutable invocado por el cargador no es el punto de entrada principal proporcionado por el programador de la aplicación, sino que es el tiempo de ejecución proporcionado por el compilador y [el enlazador](#) que crea el ejecutable. El Runtime configura el entorno para la aplicación y luego llama al punto de entrada principal provisto por el programador.

Una aplicación de consola de Windows puede tener varias interfaces ligeramente diferentes para el punto de entrada principal proporcionado por el programador. La diferencia entre estos es si el punto de entrada principal es el `int main (int argc, char *argv[])` tradicional `int main (int argc,`

`char *argv[]`) o si es la versión específica de Windows de `int _tmain(int argc, _TCHAR* argv[])` que proporciona una amplia Caracteres en los parámetros de la aplicación. Si genera un proyecto de aplicación de consola Windows Win32 utilizando Visual Studio, la fuente generada será la versión específica de Windows.

Una aplicación de Windows (GUI) tiene una interfaz diferente para el punto de entrada principal provisto por el programador. Este punto de entrada principal proporcionado por el programador tiene una interfaz más compleja porque el Runtime configura un entorno de GUI y proporciona información adicional junto con los parámetros de la aplicación.

Este ejemplo explica la interfaz del punto de entrada principal de la ventana de Windows (GUI). Para explorar estos temas debes tener:

- un IDE con compilador (preferiblemente Visual Studio)
- Conocimiento c

Cree un proyecto de Windows Win32 vacío (GUI, no consola) usando el IDE. La configuración del proyecto debe establecerse para una aplicación de ventana (no una aplicación de consola) para que el vinculador se vincule con el tiempo de ejecución correcto. Cree un archivo `main.c` agregándolo al proyecto y luego escriba el siguiente código:

```
#include <windows.h>

int APIENTRY WinMain(HINSTANCE hInst, HINSTANCE hInstPrev, PSTR cmdline, int cmdshow)
{
    return MessageBox(NULL, "hello, world", "caption", 0);
}
```

Este es nuestro programa Win32 "Hola, mundo". El primer paso es incluir los archivos de encabezado de Windows. El encabezado principal para todo Windows es `windows.h`, pero hay otros.

El `WinMain` es diferente de un `int main()` estándar usado con una aplicación de consola. Hay más parámetros utilizados en la interfaz y, lo que es más importante, el punto de entrada principal para una aplicación de ventana utiliza una convención de llamada diferente de la estándar C / C++.

El calificador `APIENTRY` indica la convención de llamada, que es el orden en el que los argumentos se insertan en la pila [†]. De forma predeterminada, la convención de llamada es la convención estándar de C indicada por `__cdecl`. Sin embargo, Microsoft utiliza un tipo diferente de convención de llamada, la convención PASCAL, para las funciones de la API de Windows que se indica con el calificador `__stdcall`. `APIENTRY` es un nombre definido para `__stdcall` en uno de los archivos de encabezado incluidos por `windows.h` (vea también [¿Qué es __stdcall?](#)).

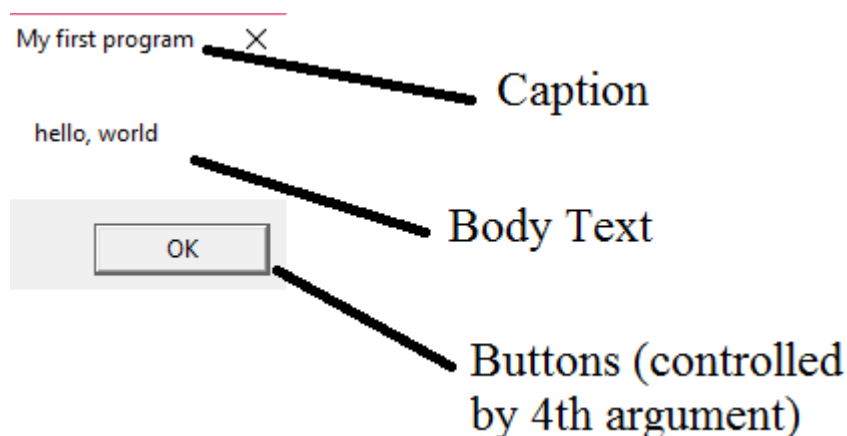
Los siguientes argumentos para `WinMain` son los siguientes:

- `hInst`: el manejador de instancia
- `hInstPrev`: el identificador de instancia anterior. Ya no se utiliza.
- `cmdline`: argumentos de la línea de comandos (vea [Pasar los argumentos de WinMain \(o wWinMain\) a la principal normal](#))

- `cmdshow`: indica si se debe mostrar una ventana.

Aún no usamos ninguno de estos argumentos.

Dentro de `WinMain()`, hay una llamada a `MessageBox()`, que muestra un diálogo simple con un mensaje, un cuadro de mensaje. El primer argumento es el manejador de la ventana del propietario. Como todavía no tenemos nuestra propia ventana, pase `NULL`. El segundo argumento es el texto del cuerpo. El tercer argumento es el título, y el cuarto argumento contiene las banderas. Cuando se pasa 0, se muestra un cuadro de mensaje predeterminado. El siguiente diagrama disecciona el cuadro de diálogo del cuadro de mensaje.



Buenos enlaces:

- [Tutorial en winprog.org](http://winprog.org)
- [Documentación de la función `MessageBox` en MSDN](#)

[†] Solo en sistemas de 32 bits. Otras arquitecturas tienen diferentes convenciones de convocatoria.

Lea Comenzando con la API de Win32 en línea:

<https://riptutorial.com/es/winapi/topic/1149/comenzando-con-la-api-de-win32>

Capítulo 2: Funciones de caracteres ansi- y anchos.

Examples

Introducción

La documentación de la API de Windows para funciones que toman una o más *cadenas* como argumento generalmente se verá así:

```
BOOL WINAPI CopyFile(  
    _In_ LPCTSTR lpExistingFileName,  
    _In_ LPCTSTR lpNewFileName,  
    _In_ BOOL bFailIfExists  
);
```

El tipo de datos para los dos parámetros de cadena se compone de varias partes:

- LP = puntero largo
- C = const
- T = TCHAR
- STR = cadena

Ahora, ¿qué significa `TCHAR` ? Esto depende de la plataforma elegida para la compilación del programa.

`CopyFile` propio `CopyFile` es solo una macro, definió algo como esto:

```
#ifdef UNICODE  
#define CopyFile CopyFileW  
#else  
#define CopyFile CopyFileA  
#endif
```

Entonces, en realidad hay dos funciones `CopyFile` y, según las banderas del compilador, la macro `CopyFile` se resolverá en una u otra.

Allí token de núcleo, `TCHAR` se define como:

```
#ifdef _UNICODE  
typedef wchar_t TCHAR;  
#else  
typedef char TCHAR;  
#endif
```

Así que de nuevo, dependiendo de las banderas de compilación, `TCHAR` es un carácter "estrecho" o "ancho" (2 bytes).

Por lo tanto, cuando se define UNICODE, `CopyFile` se define como `CopyFileW`, que usará matrices de caracteres de 2 bytes como su parámetro, que se espera que esté codificado en UTF-16.

Si no se define UNICODE, `CopyFile` se define como `CopyFileA` que utiliza matrices de caracteres de un solo byte que se espera que estén codificadas en la codificación ANSI predeterminada del usuario actual.

Hay dos macros similares: `UNICODE` hace que las API de Windows esperen cadenas anchas y `_UNICODE` (con un guión bajo) que habilita funciones similares en la biblioteca de tiempo de ejecución de C.

Estas definiciones nos permiten escribir código que se compila tanto en ANSI como en modo Unicode.

Es importante saber que la codificación ANSI puede ser una codificación de un solo byte (es decir, latin-1) y una codificación de múltiples bytes (es decir, shift jis), aunque desafortunadamente no se admite el utf-8.

Esto significa que no se puede asumir que ni la variante ANSI ni la variante de caracteres anchos de estas funciones funcionen con codificaciones de ancho fijo.

Lea [Funciones de caracteres ansi- y anchos](https://riptutorial.com/es/winapi/topic/2450/funciones-de-caracteres-ansi-y-anchos). en línea:

[https://riptutorial.com/es/winapi/topic/2450/funciones-de-caracteres-ansi-y-anchos-](https://riptutorial.com/es/winapi/topic/2450/funciones-de-caracteres-ansi-y-anchos)

Capítulo 3: Gestión de archivos

Examples

Crea un archivo y escríbelo

Este ejemplo crea un nuevo archivo llamado "NewFile.txt", luego escribe "Hello World!" a su cuerpo. Si el archivo ya existe, `CreateFile` fallará y no se escribirán datos. Vea el parámetro `dwCreationDisposition` en la [documentación de CreateFile](#) si no desea que la función falle si el archivo ya existe.

```
#include <Windows.h>
#include <string>

int main()
{
    // Open a handle to the file
    HANDLE hFile = CreateFile(
        L"C:\\NewFile.txt",    // Filename
        GENERIC_WRITE,        // Desired access
        FILE_SHARE_READ,      // Share mode
        NULL,                 // Security attributes
        CREATE_NEW,           // Creates a new file, only if it doesn't already exist
        FILE_ATTRIBUTE_NORMAL, // Flags and attributes
        NULL);               // Template file handle

    if (hFile == INVALID_HANDLE_VALUE)
    {
        // Failed to open/create file
        return 2;
    }

    // Write data to the file
    std::string strText = "Hello World!"; // For C use LPSTR (char*) or LPWSTR (wchar_t*)
    DWORD bytesWritten;
    WriteFile(
        hFile,                // Handle to the file
        strText.c_str(),      // Buffer to write
        strText.size(),       // Buffer size
        &bytesWritten,        // Bytes written
        nullptr);            // Overlapped

    // Close the handle once we don't need it.
    CloseHandle(hFile);
}
```

Referencia API:

- [MSDN CreateFile](#)
- [MSDN WriteFile](#)

Lea Gestión de archivos en línea: <https://riptutorial.com/es/winapi/topic/1765/gestion-de-archivos>

Capítulo 4: Gestión de procesos y subprocessos

Examples

Crea un proceso y comprueba su código de salida.

Este ejemplo inicia el Bloc de notas, espera a que se cierre y luego obtiene su código de salida.

```
#include <Windows.h>

int main()
{
    STARTUPINFO si = { 0 };
    si.cb = sizeof(si);
    PROCESS_INFORMATION pi = { 0 };

    // Create the child process
    BOOL success = CreateProcessW(
        L"C:\\Windows\\system32\\notepad.exe", // Path to executable
        NULL, // Command line arguments
        NULL, // Process attributes
        NULL, // Thread attributes
        FALSE, // Inherit handles
        0, // Creation flags
        NULL, // Environment
        NULL, // Working directory
        &si, // Startup info
        &pi); // Process information

    if (success)
    {
        // Wait for the process to exit
        WaitForSingleObject(pi.hProcess, INFINITE);

        // Process has exited - check its exit code
        DWORD exitCode;
        GetExitCodeProcess(pi.hProcess, &exitCode);

        // At this point exitCode is set to the process' exit code

        // Handles must be closed when they are no longer needed
        CloseHandle(pi.hThread);
        CloseHandle(pi.hProcess);
    }
}
```

Referencias (MSDN):

- [CreateProcess](#)
- [WaitForSingleObject](#)
- [GetExitCodeProcess](#)
- [CloseHandle](#)

Crear un nuevo hilo

```
#include <Windows.h>

DWORD WINAPI DoStuff(LPVOID lpParameter)
{
    // The new thread will start here
    return 0;
}

int main()
{
    // Create a new thread which will start at the DoStuff function
    HANDLE hThread = CreateThread(
        NULL,          // Thread attributes
        0,             // Stack size (0 = use default)
        DoStuff,        // Thread start address
        NULL,          // Parameter to pass to the thread
        0,             // Creation flags
        NULL);         // Thread id
    if (hThread == NULL)
    {
        // Thread creation failed.
        // More details can be retrieved by calling GetLastError()
        return 1;
    }

    // Wait for thread to finish execution
    WaitForSingleObject(hThread, INFINITE);

    // Thread handle must be closed when no longer needed
    CloseHandle(hThread);

    return 0;
}
```

Tenga en cuenta que el CRT también proporciona las API `_beginthread` y `_beginthreadex` para crear subprocesos, que no se muestran en este ejemplo. El siguiente enlace explica [las diferencias entre estas API y la API `CreateThread`](#).

Referencias (MSDN):

- [CreateThread](#)
- [WaitForSingleObject](#)
- [CloseHandle](#)
- [_beginthread , _beginthreadex](#)

Lea [Gestión de procesos y subprocesos en línea](#):

<https://riptutorial.com/es/winapi/topic/1756/gestion-de-procesos-y-subprocesos>

Capítulo 5: Informe de errores y manejo

Observaciones

Cada hilo tendrá su propio último código de error. La API de Windows establecerá el último código de error en el subproceso de llamada.

Siempre debe llamar a la función `GetLastError` inmediatamente después de verificar el valor de retorno de una función de la API de Windows.

La mayoría de las funciones de la API de Windows establecen el último código de error cuando fallan. Algunos también establecerán el último código de error cuando tengan éxito. Hay una serie de funciones que no establecen el último código de error. Consulte siempre la documentación de la función API de Windows.

No es seguro usar `FORMAT_MESSAGE_FROM_SYSTEM` sin `FORMAT_MESSAGE_IGNORE_INSERTS` cuando se usa la función `FormatMessage` para obtener una descripción de un código de error.

Examples

Introducción

La API de Windows se proporciona mediante una interfaz que se puede llamar en C. El éxito o el fracaso de una llamada a la API se informa estrictamente a través de valores de retorno. Las excepciones no forman parte del contrato documentado (aunque algunas **implementaciones de API** pueden generar excepciones [SEH](#) , por ejemplo, al pasar un argumento *lpCommandLine* de solo lectura a [CreateProcess](#)).

El informe de errores aproximadamente se divide en una de cuatro categorías:

- [Solo valor de retorno](#)
- [Valor de retorno con información adicional en caso de fallo.](#)
- [Valor de retorno con información adicional sobre el fracaso y el éxito](#)
- [Valor de retorno](#) `HRESULT`

La documentación para cada llamada a la API llama explícitamente, cómo se informan los errores. Siempre consulte la documentación.

Error reportado solo por el valor de retorno

Algunas llamadas a la API devuelven un solo indicador de falla / éxito, sin ninguna información adicional (por ejemplo, [GetObject](#)):

```
if ( GetObjectW( obj, 0, NULL ) == 0 ) {  
    // Failure: no additional information available.  
}
```

Error reportado con información adicional en caso de falla

Además de un valor de retorno de error / éxito, algunas llamadas a la API también establecen el último error en caso de error (por ejemplo, [CreateWindow](#)). La documentación generalmente contiene la siguiente redacción estándar para este caso:

Si la función tiene éxito, el valor de retorno es *<valor de éxito específico de la API>* .

Si la función falla, el valor de retorno es *<valor de error específico de API>* . Para obtener información de error extendida, llame a [GetLastError](#) .

```
if ( CreateWindowW( ... ) == NULL ) {  
    // Failure: get additional information.  
    DWORD dwError = GetLastError();  
} else {  
    // Success: must not call GetLastError.  
}
```

Es vital que llame a `GetLastError()` INMEDIATAMENTE. El último código de error puede ser sobrescrito por cualquier otra función, por lo que si hay una llamada de función extra entre la función que falló y la llamada a `GetLastError()` , el retorno de `GetLastError()` ya no será confiable. Tenga mucho cuidado al tratar con constructores de C ++.

Una vez que obtenga un código de error, deberá interpretarlo. Puede obtener una lista completa de códigos de error en MSDN, en la [página Códigos de error del sistema \(Windows\)](#) .

Alternativamente, puede buscar en los archivos de encabezado de su sistema; El archivo con todas las constantes de código de error es `winerror.h` . (Si tiene el SDK oficial de Microsoft para Windows 8 o más reciente, esto está en la subcarpeta `shared` de la carpeta de inclusión).

Notas sobre la llamada a `GetLastError()` en otros lenguajes de programación

Idiomas .net (C #, VB, etc.)

Con .net, no **debe** P / Invoke a `GetLastError()` directamente. Esto se debe a que el tiempo de ejecución de .net realizará otras llamadas a la API de Windows en el mismo hilo que está detrás de su espalda. Por ejemplo, el recolector de basura puede llamar a `VirtualFree()` si encuentra suficiente memoria que ya no está usando, y *esto puede suceder entre su llamada de función deseada y su llamada a `GetLastError()`* .

En su lugar, .net proporciona la función `Marshal.GetLastWin32Error()` , que recuperará el último error de la última llamada P / Invoke que usted mismo realizó. Use esto en lugar de llamar a `GetLastError()` directamente.

(.net no parece impedirle importar `GetLastError()` todos modos; no estoy seguro de por qué).

Ir

Las diversas funciones que proporciona Go para llamar a las funciones de DLL (que residen en el paquete `syscall` y en el paquete `golang.org/x/sys/windows`) devuelven tres valores: `r1` , `r2` y `err` . `r2` nunca se usa; Puede usar el identificador en blanco allí. `r1` es el valor de retorno de la función. `err` es el resultado de llamar a `GetLastError()` pero se convierte en un tipo que implementa un `error` , por lo que puede pasarlo a las funciones de llamada para que lo manejen.

Como Go no sabe cuándo llamar a `GetLastError()` y cuándo no, **siempre** devolverá un error no `nil` . Por lo tanto, el lenguaje típico de manejo de errores Go

```
r1, _, err := syscall.Syscall12(CreateWindowW.Addr(), ...)\nif err != nil {\n    // handle err\n}\n// use r1
```

no trabajará. En su lugar, debe verificar `r1` , exactamente como lo haría en C, y solo usar `err` si eso indica *que* la función devolvió un error:

```
r1, _, err := syscall.Syscall12(CreateWindowW.Addr(), ...)\nif r1 == 0 {\n    // handle err\n}\n// use r1
```

Error reportado con información adicional sobre el fracaso y el éxito

Algunas llamadas a la API pueden tener éxito o fallar de más de una manera. Las API generalmente devuelven información adicional tanto para invocaciones exitosas como para errores (por ejemplo, [CreateMutex](#)).

```
if ( CreateMutexW( NULL, TRUE, L"Global\\MyNamedMutex" ) == NULL ) {\n    // Failure: get additional information.\n    DWORD dwError = GetLastError();\n} else {\n    // Success: Determine which mutex was returned.\n    if ( GetLastError() == ERROR_ALREADY_EXISTS ) {\n        // Existing mutex object returned.\n    } else {\n        // Newly created mutex object returned.\n    }\n}
```

Error reportado como valor HRESULT

Los [HRESULT](#) son valores numéricos de 32 bits, donde los bits o rangos de bits codifican información bien definida. El MSB es un indicador de falla / éxito, con los bits restantes almacenando información adicional. Fracaso o el éxito se puede determinar usando los [fallidos](#) o [tenido éxito](#) macros. `HRESULT` se usan comúnmente con COM, pero también aparecen en implementaciones que no son COM (por ejemplo, [StringCchPrintf](#)).

```
const size_t cchBuf = 5;
```



```

wchar_t buffer[cchBuf] = { 0 };
HRESULT hr = StringCchPrintfW( buffer, cchBuf, L"%s", L"Hello, world!" );
if ( FAILED( hr ) ) {
    // Failure: Determine specific reason.
    switch ( hr ) {
    case STRSAFE_E_INSUFFICIENT_BUFFER:
        // Buffer too small; increase buffer and retry.
        ...
    case STRSAFE_E_INVALID_PARAMETER:
        // Invalid parameter; implement custom error handling (e.g. logging).
        ...
    default:
        // Some other error code; implement custom error handling (e.g. logging).
        ...
    }
}
}

```

Convertir un código de error en una cadena de mensaje

`GetLastError` devuelve un código de error numérico. Para obtener un mensaje de error descriptivo (*por ejemplo* , para mostrarlo a un usuario), puede llamar a `FormatMessage` :

```

// This functions fills a caller-defined character buffer (pBuffer)
// of max length (cchBufferLength) with the human-readable error message
// for a Win32 error code (dwErrorCode).
//
// Returns TRUE if successful, or FALSE otherwise.
// If successful, pBuffer is guaranteed to be NUL-terminated.
// On failure, the contents of pBuffer are undefined.
BOOL GetErrorMessage(DWORD dwErrorCode, LPTSTR pBuffer, DWORD cchBufferLength)
{
    if (cchBufferLength == 0)
    {
        return FALSE;
    }

    DWORD cchMsg = FormatMessage(FORMAT_MESSAGE_FROM_SYSTEM | FORMAT_MESSAGE_IGNORE_INSERTS,
                                NULL, /* (not used with FORMAT_MESSAGE_FROM_SYSTEM) */
                                dwErrorCode,
                                MAKELANGID(LANG_NEUTRAL, SUBLANG_DEFAULT),
                                pBuffer,
                                cchBufferLength,
                                NULL);

    return (cchMsg > 0);
}

```

En **C++** , puede simplificar considerablemente la interfaz utilizando la clase `std::string` :

```

#include <Windows.h>
#include <exception>
#include <stdexcept>
#include <memory>
#include <string>
typedef std::basic_string<TCHAR> String;

```

```
String GetErrorMessage(DWORD dwErrorCode)
{
    LPTSTR psz = NULL;
    const DWORD cchMsg = FormatMessage(FORMAT_MESSAGE_FROM_SYSTEM
                                      | FORMAT_MESSAGE_IGNORE_INSERTS
                                      | FORMAT_MESSAGE_ALLOCATE_BUFFER,
    NULL, // (not used with FORMAT_MESSAGE_FROM_SYSTEM)
    dwErrorCode,
    MAKELANGID(LANG_NEUTRAL, SUBLANG_DEFAULT),
    reinterpret_cast<LPTSTR>(&psz),
    0,
    NULL);

    if (cchMsg > 0)
    {
        // Assign buffer to smart pointer with custom deleter so that memory gets released
        // in case String's c'tor throws an exception.
        auto deleter = [](void* p) { ::HeapFree(::GetProcessHeap(), 0, p); };
        std::unique_ptr<TCHAR, decltype(deleter)> ptrBuffer(psz, deleter);
        return String(ptrBuffer.get(), cchMsg);
    }
    else
    {
        throw std::runtime_error("Failed to retrieve error message string.");
    }
}
```

NOTA: Estas funciones también funcionan para los [valores HRESULT](#) . Simplemente cambie el primer parámetro de `DWORD dwErrorCode` a `HRESULT hResult` . El resto del código puede permanecer sin cambios.

Lea Informe de errores y manejo en línea: <https://riptutorial.com/es/winapi/topic/2573/informe-de-errores-y-manejo>

Capítulo 6: Mensajes de ventana

Sintaxis

- `#include <windows.h>`
- `BOOL WINAPI DestroyWindow (HWND hwnd);`
- `VOID WINAPI PostQuitMessage (int exitcode);`
- `BOOL WINAPI MoveWindow (HWND hwnd, int x, int y, int cx, int cy, BOOL bRepaint);`

Examples

WM_CREATE

Se envía un mensaje WM_CREATE a su procedimiento de ventana durante la llamada a `CreateWindowEx` la ventana. El argumento `lp` contiene un puntero a un `CREATESTRUCT` que contiene los argumentos pasados a `CreateWindowEx`. Si una aplicación devuelve 0 de WM_CREATE, se crea la ventana. Si una aplicación devuelve -1, la creación se cancela.

```
LRESULT CALLBACK winproc(HWND hwnd, UINT wm, WPARAM wp, LPARAM lp)
{
    switch (wm) {
        case WM_CREATE:
            CREATESTRUCT *cs = (CREATESTRUCT *) lp;
            if (MessageBox(hwnd,
                "Do you want to continue creating the window?", "", MB_YESNO)
                == IDYES) {
                /* create window controls */
                return 0;
            }
            /* cancel creation */
            return -1;
    }
    return DefWindowProc(hwnd, wm, wp, lp);
}
```

WM_DESTROY

Este mensaje se envía a su procedimiento de ventana cuando se destruye una ventana. Se envía una vez que la ventana se elimina de la pantalla. La mayoría de las aplicaciones liberan todos los recursos, como la memoria o los controladores, obtenidos en WM_CREATE. Si maneja este mensaje, devuelva 0.

```
LRESULT CALLBACK winproc(HWND hwnd, UINT wm, WPARAM wp, LPARAM lp)
{
    static char *text;
    switch (wm) {
        case WM_DESTROY:
            text = malloc(256);
            /* use the allocated memory */
    }
}
```

```

        return 0;
    case WM_CLOSE:
        switch (MessageBox(hwnd, "Save changes?", "", MB_YESNOCANCEL)) {
            case IDYES:
                savedoc();

                /* fall through */

            case IDNO:
                DestroyWindow(hwnd);
                break;
        }
        return 0;
    case WM_DESTROY:
        /* free the memory */
        free(text);
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProc(hwnd, wm, wp, lp);
}

```

WM_CLOSE

Se envía cuando se hace clic en el botón de cierre de una aplicación. No confunda esto con `WM_DESTROY` que se envía cuando se destruye una ventana. La principal diferencia radica en el hecho de que el cierre se puede cancelar en `WM_CLOSE` (piense en que Microsoft Word le pide que guarde sus cambios), en lugar de destruir cuando la ventana ya se ha cerrado (piense en la memoria liberadora de Microsoft Word).

```

LRESULT CALLBACK winproc(HWND hwnd, UINT wm, WPARAM wp, LPARAM lp)
{
    static char *text;
    switch (wm) {
        case WM_CREATE:
            text = malloc(256);
            /* use the allocated memory */
            return 0;
        case WM_CLOSE:
            switch (MessageBox(hwnd, "Save changes?", "", MB_YESNOCANCEL)) {
                case IDYES:
                    savedoc();

                    /* fall through */

                case IDNO:
                    DestroyWindow(hwnd);
                    break;
            }
            return 0;
        case WM_DESTROY:
            /* free the memory */
            free(text);
            PostQuitMessage(0);
            return 0;
    }
    return DefWindowProc(hwnd, wm, wp, lp);
}

```

WM_SIZE

Este mensaje se envía al procedimiento de la ventana de la ventana después de que su tamaño haya cambiado. La razón más común para manejar este mensaje es ajustar la posición de cualquier ventana secundaria. Por ejemplo, en el Bloc de notas, cuando se cambia el tamaño de la ventana, también se cambia el tamaño de la ventana secundaria (control de edición). Devuelve 0 si manejas este mensaje.

Argumento	Valor
wp	Una de las constantes de tamaño de ventana .
lp	LOWORD (lp) es el nuevo ancho del área del cliente HIWORD (lp) es la nueva altura del área del cliente.

```
LRESULT CALLBACK winproc(HWND hwnd, UINT wm, WPARAM wp, LPARAM lp)
{
    switch (wm) {
        case WM_SIZE:
            /* hwndEdit is the handle of the edit control window */
            MoveWindow(hwndEdit, 0, 0, LOWORD(lp), HIWORD(lp), TRUE);
            return 0;
    }
    return DefWindowProc(hwnd, wm, wp, lp);
}
```

WM_COMMAND

Enviado a un procedimiento de ventana cuando:

- El usuario selecciona un elemento de un menú.
- un control envía una notificación a su ventana principal
- una pulsación del acelerador se traduce

Fuente del mensaje	HIWORD (wp)	LOWORD (wp)	lp
Menú	0	ID de menú (IDM_*)	0
Acelerador	1	ID de aceleración (IDM_*)	0
Controlar	código de notificación	ID de control	HWND de ventana de control

Por ejemplo, en el Bloc de notas, cuando un usuario hace clic en "Archivo-> Abrir", se muestra un cuadro de diálogo para permitirle al usuario abrir un archivo. Los ítems del menú se procesan en el mensaje WM_CREATE del procedimiento de la ventana de esta manera:

```
LRESULT CALLBACK winproc(HWND hwnd, UINT wm, WPARAM wp, LPARAM lp)
{
```

```
switch (wm) {  
    case WM_COMMAND:  
        switch (LOWORD(wp)) {  
            case ID_FILE_OPEN:  
                /* show file open dialog */  
                break;  
            case ID_FILE_NEW:  
                /* create new instance */  
                break;  
        }  
        return 0;  
    }  
    return DefWindowProc(hwnd, wm, wp, lp);  
}
```

Lea Mensajes de ventana en línea: <https://riptutorial.com/es/winapi/topic/2449/mensajes-de-ventana>

Capítulo 7: Servicios de Windows

Examples

Compruebe si un servicio está instalado

Este ejemplo muestra cómo puede verificar si un servicio ya existe (*es decir* , está instalado en la máquina) o no. Este código solo requiere los privilegios más bajos necesarios, por lo que cada proceso puede realizar la verificación, sin importar en qué nivel de seguridad se esté ejecutando.

```
#define UNICODE
#define _UNICODE
#include <Windows.h>
#include <string>
#include <iostream>

enum Result
{
    unknown,
    serviceManager_AccessDenied,
    serviceManager_DatabaseDoesNotExist,
    service_AccessDenied,
    service_InvalidServiceManagerHandle,
    service_InvalidServiceName,
    service_DoesNotExist,
    service_Exist
};

Result ServiceExists(const std::wstring &serviceName)
{
    Result r = unknown;

    // Get a handle to the SCM database
    SC_HANDLE manager = OpenSCManager(NULL, SERVICES_ACTIVE_DATABASE, GENERIC_READ);

    if (manager == NULL)
    {
        DWORD lastError = GetLastError();

        // At this point, we can return directly because no handles need to be closed.
        if (lastError == ERROR_ACCESS_DENIED)
            return serviceManager_AccessDenied;
        else if (lastError == ERROR_DATABASE_DOES_NOT_EXIST)
            return serviceManager_DatabaseDoesNotExist;
        else
            return unknown;
    }

    SC_HANDLE service = OpenService(manager, serviceName.c_str(), GENERIC_READ);

    if (service == NULL)
    {
        DWORD error = GetLastError();

        if (error == ERROR_ACCESS_DENIED)
            r = service_AccessDenied;
```

```

        else if (error == ERROR_INVALID_HANDLE)
            r = service_InvalidServiceManagerHandle;
        else if (error == ERROR_INVALID_NAME)
            r = service_InvalidServiceName;
        else if (error == ERROR_SERVICE_DOES_NOT_EXIST)
            r = service_DoesNotExist;
        else
            r = unknown;
    }
    else
        r = service_Exist;

    if (service != NULL)
        CloseServiceHandle(service);

    if (manager != NULL)
        CloseServiceHandle(manager);

    return r;
}

int main()
{
    std::wstring serviceName = L"MSSQL$SQLEXPRESS"; // name of the service to check
    Result result = ServiceExists(serviceName);
    if (result == service_Exist)
        std::wcout << L"The service '" << serviceName << "' exists." << std::endl;
    else if (result == service_DoesNotExist)
        std::wcout << L"The service '" << serviceName << "' does not exist." << std::endl;
    else
        std::wcout << L"An error has occurred, and it could not be determined whether the
service '" << serviceName << "' exists or not." << std::endl;
}

```

Referencia API:

- [MSDN OpenSCManager](#)
- [MSDN OpenService](#)
- [MSDN CloseServiceHandle](#)

Lea Servicios de Windows en línea: <https://riptutorial.com/es/winapi/topic/2256/servicios-de-windows>

Capítulo 8: Subclase de Windows

Introducción

La subclasificación de ventanas es una forma de conectarse al procedimiento de ventana estándar y modificar o ampliar su comportamiento predeterminado. Una aplicación asigna subclases a una ventana reemplazando el procedimiento de ventana original de la ventana con un nuevo procedimiento de ventana. Este nuevo procedimiento de ventana recibe cualquier mensaje enviado o publicado en la ventana.

Sintaxis

- `BOOL SetWindowSubclass (HWND hWnd, SUBCLASSPROC SubclassProc, UINT_PTR SubclassId, DWORD_PTR RefData);`
- `BOOL RemoveWindowSubclass (HWND hWnd, SUBCLASSPROC SubclassProc, UINT_PTR SubclassId);`
- `BOOL GetWindowSubclass (HWND hWnd, SUBCLASSPROC SubclassProc, UINT_PTR SubclassId, DWORD_PTR * RefData);`
- `LRESULT DefSubclassProc (HWND hWnd, UINT Msg, WPARAM wParam, LPARAM lParam);`

Parámetros

Parámetro	Detalle
hWnd	El manejador de la ventana a subclase.
SubclassProc	El procedimiento de devolución de llamada de subclase.
Subclase	El ID especificado por el usuario para identificar la subclase, junto con el procedimiento de la subclase identifica de forma única una subclase. Simplemente puede ser un número consecutivo arbitrario.
Datos de referencia	Datos especificados por el usuario. El significado está determinado por la aplicación. Se pasa a la devolución de llamada de la subclase en forma no modificada. Podría ser un puntero de objeto a una instancia de clase, por ejemplo.

Observaciones

Documentación MSDN

- [Acerca de los procedimientos de Windows](#)
- [Controles de subclases](#)

Examples

Subclasificar el control de botones de ventanas dentro de la clase C ++

Este ejemplo muestra cómo manipular el tamaño ideal del botón especificando un tamaño fijo.

```
class ButtonSubclass {
public:

    ButtonSubclass(HWND hWndButton) {
        SetWindowSubclass(hWndButton, MyButtonSubclassProc, 1, (DWORD_PTR) this);
    }
    ~ButtonSubclass() {
        RemoveWindowSubclass(hWndButton, MyButtonSubclassProc, 1, (DWORD_PTR) this);
    }

protected:

    static LRESULT CALLBACK MyButtonSubclassProc(
        HWND hWnd, UINT Msg, WPARAM w, LPARAM l, DWORD_PTR RefData) {

        ButtonSubclass* o = reinterpret_cast<ButtonSubclass*>(RefData);

        if (Msg == BCM_GETIDEALSIZE) {
            reinterpret_cast<SIZE*>(lParam)->cx = 100;
            reinterpret_cast<SIZE*>(lParam)->cy = 100;
            return TRUE;
        }
        return DefSubclassProc(hWnd, Msg, w, l);
    }
}
```

Procedimiento de instalación y eliminación de subclases.

Los siguientes métodos instalan o eliminan la devolución de llamada de la subclase. La combinación de `SubclassId` y `SubclassProc` identifica de forma única una subclase. No hay un recuento de referencias, llamar a `SetWindowSubclass` varias veces con diferentes `RefData` solo actualiza ese valor, pero no hace que la devolución de llamada de la subclase se llame varias veces.

```
BOOL SetWindowSubclass(HWND hWnd, SUBCLASSPROC SubclassProc, UINT_PTR SubclassId, DWORD_PTR
RefData);
BOOL RemoveWindowSubclass(HWND hWnd, SUBCLASSPROC SubclassProc, UINT_PTR SubclassId);
```

Para recuperar los datos de referencia que se pasaron en la última llamada a `SetWindowSubclass`, se puede usar el método `GetWindowSubclass`.

```
BOOL GetWindowSubclass(HWND hWnd, SUBCLASSPROC SubclassProc, UINT_PTR SubclassId, DORD_PTR*
RefData);
```

Parámetro	Detalle
hWnd	El manejador de la ventana a subclase.

Parámetro	Detalle
SubclassProc	El procedimiento de devolución de llamada de subclase.
Subclase	El ID especificado por el usuario para identificar la subclase, junto con el procedimiento de la subclase identifica de forma única una subclase. Simplemente puede ser un número consecutivo arbitrario.
Datos de referencia	Datos especificados por el usuario. El significado está determinado por la aplicación. Se pasa a la devolución de llamada de la subclase en forma no modificada. Podría ser un puntero de objeto a una instancia de clase, por ejemplo.

La devolución de llamada de la subclase es responsable de llamar al siguiente controlador en la cadena de subclases de la ventana. `DefSubclassProc` llama al siguiente controlador en la cadena de subclases de la ventana. El último controlador llama al procedimiento de ventana original. Debe llamarse en cualquier procedimiento de devolución de llamada de subclases, a menos que la aplicación maneje completamente el mensaje.

```
LRESULT DefSubclassProc(HWND hWnd, UINT Msg, WPARAM wParam, LPARAM lParam);
```

Parámetro	Detalle
hWnd	Manejador de ventana desde donde se origina el mensaje
Msg	Mensaje de ventana
wParam	Argumento WPARAM, este valor depende del mensaje de ventana específico
Param	Argumento LPARAM, este valor depende del mensaje de ventana específico

SUBCLASSPROC

Es similar a la `WINDOWPROC` llamada de `WINDOWPROC` pero contiene un argumento adicional `RefData`.

```
typedef LRESULT (CALLBACK *SUBCLASSPROC) (
    HWND hWnd,
    UINT Msg,
    WPARAM wParam,
    LPARAM lParam,
    UINT_PTR SubclassId,
    DWORD_PTR RefData
);
```

Manejo de mensajes de notificación de controles comunes dentro de la clase C++

```
class MyToolbarControl {
public:
```

```

MyToolbarControl(HWND hWndToolbar, HWND hWndNotifyParent = nullptr) : _Handle(hWndToolbar)
{
    if (hWndNotifyParent == nullptr) {
        hWndNotifyParent = GetAncestor(hWndToolbar, GA_ROOTOWNER);
    }
    SetWindowSubclass(
        hWndNotifyParent, SubclassWindowProc, reinterpret_cast<UINT_PTR>(this),
        reinterpret_cast<DWORD_PTR>(this)
    );
}
~MyToolbarControl() {
    RemoveWindowSubclass(
        hWndNotifyParent, SubclassWindowProc, reinterpret_cast<UINT_PTR>(this),
        reinterpret_cast<DWORD_PTR>(this)
    );
}

protected:
    HWND _Handle;

    static LRESULT CALLBACK SubclassWindowProc(
        HWND hWnd, UINT Msg, WPARAM w, LPARAM l, UINT_PTR SubclassId, DWORD_PTR RefData) {
        MyToolbarControl * w = reinterpret_cast<MyToolbarControl *>(RefData);
        if (Msg == WM_NOTIFY) {
            NMHDR* h = reinterpret_cast<NMHDR*>(l);
            if (h->hwndFrom == w->_Handle) {
                // Handle notification message here...
            }
        }
        return DefSubclassProc(hWnd, Msg, w, l);
    }
};

```

Lea Subclase de Windows en línea: <https://riptutorial.com/es/winapi/topic/9399/subclase-de-windows>

Capítulo 9: Tratar con ventanas

Examples

Creando una ventana

```
#define UNICODE
#define _UNICODE
#include <windows.h>
#include <tchar.h>
const TCHAR CLSNAME[] = TEXT("helloworldWClass");
LRESULT CALLBACK winproc(HWND hwnd, UINT wm, WPARAM wp, LPARAM lp);

int WINAPI WinMain(HINSTANCE hInst, HINSTANCE hPrevInst, PTSTR cmdline,
                  int cmdshow)
{
    WNDCLASSEX wc = { };
    MSG msg;
    HWND hwnd;

    wc.cbSize      = sizeof (wc);
    wc.style       = 0;
    wc.lpfnWndProc = winproc;
    wc.cbClsExtra  = 0;
    wc.cbWndExtra  = 0;
    wc.hInstance   = hInst;
    wc.hIcon       = LoadIcon (NULL, IDI_APPLICATION);
    wc.hCursor     = LoadCursor (NULL, IDC_ARROW);
    wc.hbrBackground = (HBRUSH) GetStockObject (WHITE_BRUSH);
    wc.lpszMenuName = NULL;
    wc.lpszClassName = CLSNAME;
    wc.hIconSm     = LoadIcon (NULL, IDI_APPLICATION);

    if (!RegisterClassEx(&wc)) {
        MessageBox(NULL, TEXT("Could not register window class"),
                  NULL, MB_ICONERROR);
        return 0;
    }

    hwnd = CreateWindowEx(WS_EX_LEFT,
                        CLSNAME,
                        NULL,
                        WS_OVERLAPPEDWINDOW,
                        CW_USEDEFAULT,
                        CW_USEDEFAULT,
                        CW_USEDEFAULT,
                        CW_USEDEFAULT,
                        NULL,
                        NULL,
                        hInst,
                        NULL);

    if (!hwnd) {
        MessageBox(NULL, TEXT("Could not create window"), NULL, MB_ICONERROR);
        return 0;
    }

    ShowWindow(hwnd, cmdshow);
```

```

UpdateWindow(hwnd);
while (GetMessage(&msg, NULL, 0, 0)) {
    TranslateMessage(&msg);
    DispatchMessage(&msg);
}
return msg.wParam;
}
LRESULT CALLBACK winproc(HWND hwnd, UINT wm, WPARAM wp, LPARAM lp)
{
    return DefWindowProc(hwnd, wm, wp, lp);
}

```

Lo primero que se ve son las dos definiciones de macro, `UNICODE` y `_UNICODE`. Estas macros hacen que nuestro programa comprenda cadenas de caracteres anchos (`wchar_t[n]`), no cadenas simples y estrechas (`char[n]`). Como resultado, todos los literales de cadena deben estar envueltos en un `TEXT()` macro. El tipo de carácter genérico para cadenas de Win32 es `TCHAR`, cuya definición depende de si se define o no `UNICODE`. Se incluye un nuevo encabezado: `<tchar.h>` contiene la Declaración de `TCHAR`.

Una ventana consiste en lo que se conoce como una *clase de ventana*. Esto describe información sobre una ventana que se compartirá entre las instancias de ella, como el icono, el cursor y otros. Una clase de ventana se identifica con un nombre de clase de ventana, que se proporciona en la variable global `CLSNAME` en este ejemplo. El primer acto de `WinMain` es completar la estructura de clase de la ventana, `WNDCLASSEX wc`. Los miembros son:

- `cbSize`: el tamaño, en bytes, de la estructura
- `style`: los estilos de clase de ventana. Esto es 0 por ahora.
- `lpfnWndProc`: Este es uno de los campos más importantes. Almacena la dirección del *procedimiento de ventana*. El procedimiento de ventana es una función que maneja eventos para todas las ventanas que son instancias de esta clase de ventana.
- `cbClsExtra`: el número de bytes adicionales para asignar a la clase de ventana. Para la mayoría de las situaciones, este miembro es 0.
- `cbWndExtra`: el número de bytes adicionales para asignar para cada ventana individual. No confunda esto con `cbClsExtra`, que es común a todas las instancias. Esto es a menudo 0.
- `hInstance`: El manejador de instancia. Basta con asignar el `hInst` argumento en `WinMain` a este campo.
- `hIcon`: El manejador de icono para la clase de ventana. `LoadIcon(NULL, IDI_APPLICATION)` carga el icono de la aplicación predeterminada.
- `hCursor`: El controlador de cursor para la clase de ventana. `LoadCursor(NULL, IDC_ARROW)` carga el cursor predeterminado.
- `hbrBackground`: Un identificador para el pincel de fondo. `GetStockObject(WHITE_BRUSH)` otorga un identificador a un pincel blanco. El valor de retorno se debe convertir porque `GetStockObject` devuelve un objeto genérico.
- `lpszMenuName`: El nombre de recurso de la barra de menú a usar. Si no se necesita una barra de menú, este campo puede ser `NULL`.
- `lpszClassName`: el nombre de clase que identifica esta estructura de clase de ventana. En este ejemplo, la variable global `CLSNAME` almacena el nombre de la clase de ventana.
- `hIconSm`: un identificador para el icono de clase pequeña.

Una vez que se inicializa esta estructura, se llama a la función `RegisterClassEx` . Esto hace que la clase de ventana se registre con Windows, lo que se conoce en la aplicación. Devuelve 0 en caso de fallo.

Ahora que la clase de ventana se ha registrado, podemos mostrar la ventana utilizando `CreateWindowEx` . Los argumentos son:

- `stylesex`: Los estilos de ventana extendida. El valor predeterminado es `WS_EX_LEFT`.
- `classname`: el nombre de la clase
- `cap`: el título de la ventana, o subtítulo. En este caso, es el título que se muestra en la barra de título de una ventana.
- `estilos`: los estilos de ventana. Si desea crear una ventana de nivel superior (principal) como esta, la bandera para pasar es `WS_OVERLAPPEDWINDOW`.
- `x`: la coordenada x de la esquina superior izquierda de la ventana.
- `y`: la coordenada y de la esquina superior izquierda de la ventana
- `cx`: el ancho de la ventana
- `cy`: la altura de la ventana
- `hwndParent`: el identificador de la ventana principal. Como esta ventana es en sí misma una ventana principal, este argumento es `NULL`.
- `hMenuOrID`: si la ventana que se está creando es una ventana principal, este argumento es un identificador del menú de la ventana. No confunda esto con el menú de clase, que es `WNDCLASSEX::lpzClassName` . El menú de clase es común a todas las instancias de ventanas con el mismo nombre de clase. Este argumento, sin embargo, es específico solo para esta instancia. Si la ventana que se está creando es una ventana secundaria, este es el ID de la ventana secundaria. En este caso, estamos creando una ventana principal sin menú, por lo que se pasa `NULL`.
- `hInst`: El manejador de la instancia de la aplicación.
- `etc`: La información adicional que se pasa al procedimiento de la ventana de la ventana. Si no se va a transmitir información adicional, pase `NULL`.

Si `x` o `y` o `cx` o `cy` es `CW_USEDEFAULT` , entonces el valor de ese argumento será determinado por Windows. Eso es lo que se hace en este ejemplo.

`CreateWindowEx` devuelve el identificador a la ventana recién creada. Si la creación de la ventana falló, devolvió `NULL` .

Luego mostramos la ventana llamando a `ShowWindow` . El primer argumento para esta función es el manejador de la ventana. El segundo argumento es el estilo de presentación, que indica cómo se mostrará la ventana. La mayoría de las aplicaciones simplemente pasan el argumento `cmdshow` pasado en `WinMain` . Una vez que se muestra la ventana, debe actualizarse mediante una llamada a `UpdateWindow` . Hace que se envíe un mensaje de actualización a la ventana. Aprenderemos lo que esto significa en otro tutorial.

Ahora viene el corazón de la aplicación: La bomba de mensajes. Bombea los mensajes enviados a esta aplicación por el sistema operativo y envía los mensajes al procedimiento de la ventana. La llamada `GetMessage` devuelve un valor distinto de cero hasta que la aplicación recibe un mensaje que lo hace salir, en cuyo caso devuelve 0. El único argumento que nos concierne es el puntero a una estructura de `MSG` que se completará con información sobre el mensaje. Los otros argumentos

son todos 0.

Dentro del bucle de mensajes, `TranslateMessage` traduce los mensajes de clave virtual en mensajes de caracteres. El significado de esto, de nuevo, no es importante para nosotros. Toma un puntero a una estructura de `MSG`. La llamada que lo sigue directamente, `DispatchMessage`, envía el mensaje señalado por su argumento al procedimiento de la ventana de la ventana. Lo último que debe hacer `WinMain` es devolver un código de estado. El miembro `wParam` de la estructura `MSG` contiene este valor de retorno, por lo que se devuelve.

Pero eso es solo para la función `WinMain`. La otra función es `winproc`, el procedimiento de ventana. Manejará los mensajes para la ventana que Windows le envía. La firma para `winproc` es:

- `hwnd`: un identificador de la ventana cuyos mensajes se están procesando.
- `wm`: el identificador de mensaje de ventana
- `wp`: uno de los argumentos de información del mensaje. Esto depende del argumento `wm`
- `lp`: Uno de los argumentos de información del mensaje. Esto depende del argumento `wm`. Este argumento se suele utilizar para transmitir punteros o manejadores.

En este programa simple, nosotros no manejamos ningún mensaje nosotros mismos. Pero eso no significa que Windows tampoco lo haga. Esta es la razón por la que se debe llamar a `DefWindowProc`, que contiene el código de manejo de ventanas predeterminado. Esta función debe llamarse al final de cada procedimiento de ventana.

¿Qué es un mango?

Un *identificador* es un tipo de datos que representa un objeto único. Son punteros, pero a estructuras de datos secretas mantenidas por el sistema operativo. Los detalles de estas estructuras no necesitan preocuparnos. Todo lo que un usuario debe hacer es simplemente crear / recuperar un identificador utilizando una llamada a la API, y pasarlo a otras llamadas a la API que tomen ese tipo de identificador. El único tipo de identificador que utilizamos fue el `HWND` devuelto por `CreateWindowEx`.

Constantes

En este ejemplo, encontramos un puñado de constantes, que están en mayúsculas y comienzan con un prefijo de 2 o 3 letras. (Los tipos de Windows también están en mayúsculas)

- `IDI_APPLICATION`: El nombre del recurso que contiene el ícono de la aplicación predeterminada. Esto se usa con `LoadIcon` o `LoadImage` (`LoadIcon` en este ejemplo).
- `IDC_ARROW`: El nombre del recurso que monta el cursor de la aplicación predeterminada. Esto se usa con `LoadIcon` o `LoadImage` (`LoadIcon` en este ejemplo).
- `WHITE_BRUSH`: El nombre de un objeto de stock. Este objeto de stock es el pincel blanco.
- `MB_ICONERROR`: un indicador utilizado con `MessageBox` para mostrar un icono de error.
- `WS_EX_LEFT`: El estilo de ventana extendida predeterminado. Esto hace que la ventana tenga propiedades alineadas a la izquierda.
- `WS_OVERLAPPEDWINDOW`: un estilo de ventana que indica que la ventana debe ser una ventana principal con una barra de título, un cuadro de tamaño y otros elementos típicos de las ventanas de nivel superior.

- `CW_USEDEFAULT`: se utiliza con `cy` argumentos `x`, `y`, `cx` o `cy` `CreateWindowEx`. Hace que Windows elija un valor válido para el argumento por el que se pasó `CW_USEDEFAULT`.

Tipos de Windows

Al programar para Windows, tendrá que acostumbrarse a los tipos de Win32, que son alias para los tipos incorporados. Estos tipos están en todas las tapas. Los tipos de alias utilizados en este programa son:

- `TCHAR`: El tipo de carácter genérico. Si se define `UNICODE`, este es un `wchar_t`. Otherwise, es un `char`.
- `UINT`: Un entero sin signo. Se utiliza para representar el identificador de mensaje en procedimientos de ventana y otros fines.
- `WPARAM`: En Win16, este era un argumento `WORD` (de ahí el prefijo `w`). Sin embargo, con la introducción de Win32, este es ahora un `UINT_PTR`. Esto ilustra el punto de estos alias de Windows; Ellos están ahí para proteger a los programas del cambio.
- `LPARAM`: Este es un argumento `LONG` (`LONG_PTR` en Win64).
- `PTSTR`: La `P` significa puntero. La `T` significa carácter genérico y la `STR` significa cadena. Por lo tanto, esto es un puntero a una cadena `TCHAR`. Otros tipos de cadena incluyen:
 - `LPTSTR`: igual que `PTSTR`
 - `LPCTSTR`: Significa `const TCHAR *`
 - `PCTSTR`: Igual que `LPCTSTR`
 - `LPWSTR`: cadena ancha (`wchar_t *`)
 - `LPCWSTR`: Significa `const wchar_t *`
 - `PWSTR`: Igual que `LPWSTR`
 - y mucho más Como puede ver, los tipos de Win32 pueden ser difíciles de entender, especialmente con tantos tipos, que es un artefacto de Win16.
- `LRESULT`: este tipo se utiliza para representar el valor de retorno de los procedimientos de ventana. Suele ser un `LARGO` (de ahí la `L`).

Lea Tratar con ventanas en línea: <https://riptutorial.com/es/winapi/topic/2782/tratar-con-ventanas>

Capítulo 10: Utilizando la documentación de MSDN

Introducción

La API de Windows es muy amplia y contiene muchas características. El tamaño de la API es tal que nadie puede saberlo todo. Si bien hay muchos recursos como StackOverflow, no hay sustituto para la documentación oficial.

Observaciones

Ejemplos de documentación:

- **Descripción general del tema:** [Consideraciones de rendimiento y mejores prácticas de Desktop Window Manager](#)
- **Ejemplos:** [personalizar una miniatura icónica y un mapa de bits de vista previa en vivo](#)
- **Funciones:** [DwmSetIconicThumbnail function](#)

Examples

Tipos de documentación disponibles

La biblioteca de MSDN contiene varios tipos diferentes de documentación que se pueden usar para implementar características.

- **Visión** general de temas Estos son descripciones generales de temas destinados a proporcionar una comprensión general de una API. Estas descripciones generales a menudo también describen las mejores prácticas y las estrategias de implementación.
- **Las muestras** demuestran el uso de API particulares. Estos son generalmente muy simplificados, no necesariamente hacen la comprobación de errores y, por lo general, no usan marcos como MFC o ATL. Proporcionan un punto de partida para el uso de características.
- Detalles de **referencia** todos los elementos de cada API. Esto incluye constantes / enumeraciones, interfaces, funciones y clases.

Nota: muchos empleados de Microsoft también mantienen blogs, como [The Old New Thing](#) de Raymond Chen, que puede complementar la documentación, pero estos blogs no son un sustituto de la documentación.

Encontrar documentación para una característica

Encontrar documentación para una característica a menudo es tan simple como buscar con un buen motor de búsqueda. Si eso falla, o si no está seguro acerca de los términos específicos, el [índice API de Windows](#) puede ayudar a localizar características específicas. La documentación

para métodos, interfaces, enumeraciones y constantes generalmente se puede encontrar buscando el nombre usando un motor de búsqueda. Además, el [Centro de desarrollo de Windows](#) puede proporcionar un valioso punto de partida.

Uso de la documentación de la función

La documentación para una función se divide en varias secciones:

Visión general

Describe para qué se utiliza la función. Esta sección también mostrará información sobre si la función está depreciada o puede que no esté disponible en versiones futuras.

Sintaxis

Muestra la declaración de la función desde el encabezado de origen apropiado. Es una referencia rápida a la firma de la función.

Parámetros

Explica cada uno de los parámetros, si el parámetro es entrada o salida, y otras consideraciones importantes.

Valor de retorno

Esta sección explica el resultado de la llamada a la función, incluido cómo detectar errores y qué información adicional está disponible. (Por ejemplo, esta sección indicará explícitamente si `GetLastError` proporcionará información adicional sobre el manejo de errores).

Observaciones

Cubre toda la información adicional requerida para usar la función, como, por ejemplo, información sobre funciones de soporte, obtención de identificadores adecuados y eliminación de recursos.

Ejemplos

Si esta sección está disponible, tiene un ejemplo del uso apropiado de la función para usar como punto de partida para la implementación.

Requerimientos

Proporciona información importante sobre los requisitos previos para llamar a la función. Esta información incluye:

- **Ciente / servidor mínimo admitido** Primera versión del sistema operativo (compatible con Microsoft) para proporcionar la función.
(Tenga en cuenta que este campo es notoriamente engañoso. A menudo, las funciones son compatibles con una versión anterior del sistema operativo, pero este campo solo muestra la versión más antigua que *actualmente es compatible con Microsoft* . Por ejemplo, la función `CreateWindow` ha sido compatible desde Windows 1.0 , pero la documentación solo muestra que ha sido admitida desde Windows 2000. La versión en línea de la documentación de MSDN no indica que *ninguna* función fuera compatible con una versión de Windows anterior a 2000, aunque muchas sí lo eran. Para el desarrollo heredado, debe consultar una versión anterior de la documentación del SDK, como podría haber sido enviada en un CD-ROM de MSDN. O bien, consulte los archivos de encabezado.)
- **Encabezado** El encabezado del SDK para `#include` que contiene la declaración de función. Si la función no está disponible en un encabezado, esto mostrará información sobre el procedimiento para llamar a la función (generalmente se llama a `GetProcAddress` para realizar el enlace dinámico en tiempo de ejecución).
- **Biblioteca** El archivo de biblioteca que se pasa al vinculador para resolver las funciones exportadas.
- **DLL** El archivo (tal como se incluye con el sistema operativo) que contiene la función exportada.
- **Fin del soporte de cliente / servidor** La última versión de Windows para admitir oficialmente la API.
- **Nombres Unicode y ANSI** Para las funciones de cadena que tienen variantes Unicode y ANSI, esto enumera los nombres reales exportados para las dos funciones. Este suele ser solo el nombre de la función con un sufijo `_W` o `_A` (respectivamente).

Lea Utilizando la documentación de MSDN en línea:

<https://riptutorial.com/es/winapi/topic/8999/utilizando-la-documentacion-de-msdn>

Creditos

S. No	Capítulos	Contributors
1	Comenzando con la API de Win32	Community , IInspectable , Richard Chambers , stackptr , Stuart , theB
2	Funciones de caracteres ansi- y anchos.	Adrian McCarthy , Ajay , IInspectable , Tannin
3	Gestión de archivos	Adi Lester , Ajay , theB
4	Gestión de procesos y subprocesos	Adi Lester , Ajay , IInspectable , theB
5	Informe de errores y manejo	Ajay , andlabs , camelCase , Cody Gray , IInspectable , stackptr
6	Mensajes de ventana	stackptr
7	Servicios de Windows	Cody Gray , Mr. Gray
8	Subclase de Windows	bkausbk
9	Tratar con ventanas	stackptr
10	Utilizando la documentación de MSDN	Cody Gray , theB